Database Foundations

Concepts • Types • ACID • CAP • Choosing the Right DB

Q.E.D. — Web Dev Bootcamp | Session 04

The Problem — Why Can't We Just Use Files?

Remember our fake_db = {} ? Here's what happens in real life:

Imagine you're building an e-commerce app:

Day 1: You store orders in a JSON file. Works fine!

Day 30: 10,000 orders. File is 50MB. Searching is slow.

Day 60: Two users buy the last item at the same time. Both get it. You oversold!

Day 90: Server crashes. JSON file gets corrupted. You lose 3 days of orders.

Day 91: Your boss fires you.

This is not hypothetical. These are real problems that every company faces. Databases were invented to solve **all of them**.

Files (JSON/CSV):

- × Data lost on crash
- × Slow search at scale
- × No concurrent safety
- × No validation rules
- × No relationships

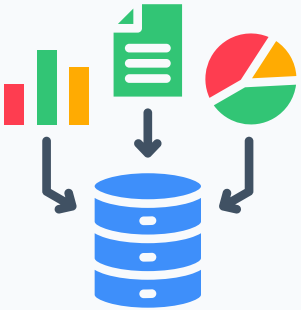
Database:

- √ Survives crashes (durability)
- √ Millisecond search (indexes)
- √ Safe concurrency (transactions)

So... What is a Database?

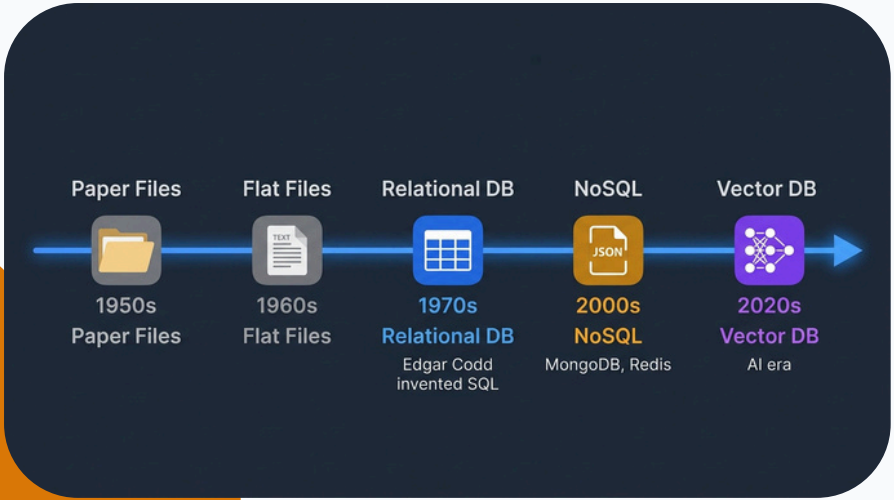
Definition:
A database is an organized system for **storing**, **retrieving**, and **managing** data efficiently — with guarantees about safety, speed, and correctness.

Feature	What it gives you
Persistence	Data survives restarts and crashes
Querying	Find any data in milliseconds
Integrity	Rules prevent invalid data
Concurrency	Multiple users, no conflicts



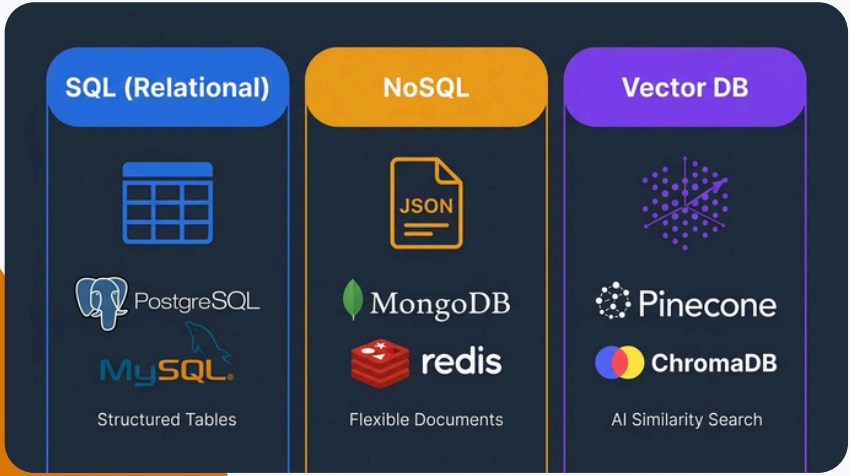
The Evolution of Data Storage

History How we got from paper files to AI-powered vector databases.



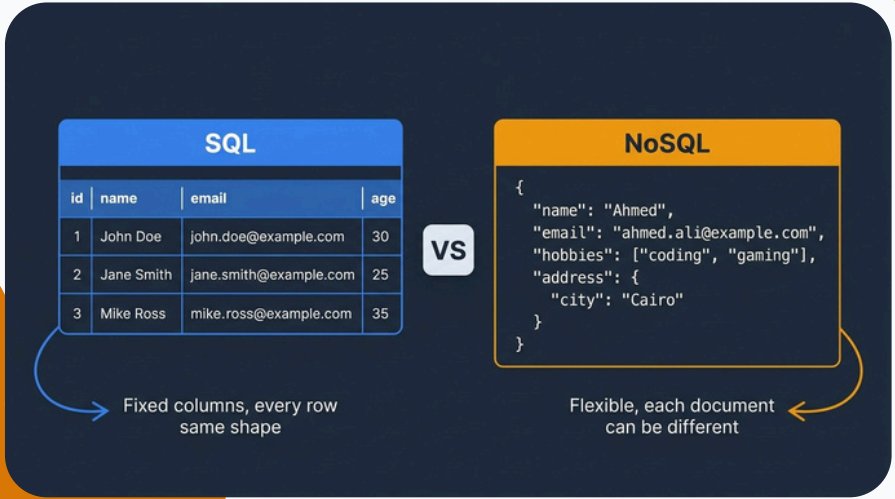
Three Types of Databases

Overview Each type is built to solve a fundamentally different problem.



SQL vs NoSQL — How Data Looks

Visual Same user data stored in two completely different ways.



SQL (Relational) Databases — Deep Dive

Type 1 The gold standard. 50+ years old and still the #1 choice worldwide.

Strengths

- **Structureddata**—Schema enforces consistency
- **Relationships**—JOINs connectt ables elegantly
- **ACIDcompliant**—Banks trustit with realmoney
- **SQLisuniversal**—Learn once,use everywhere
- **Matureecosystem**—40+yearsofbattle-tested optimization

Weaknesses

- **Rigidschema**—Mustdefinestructureupfront
- **Verticalscaling**—Biggerserver,notmoreservers
- **Schema migrations** — Altering a billion-row table is slow
- **Notidealforblobs**—Images,videos,rawlogs

When to Use

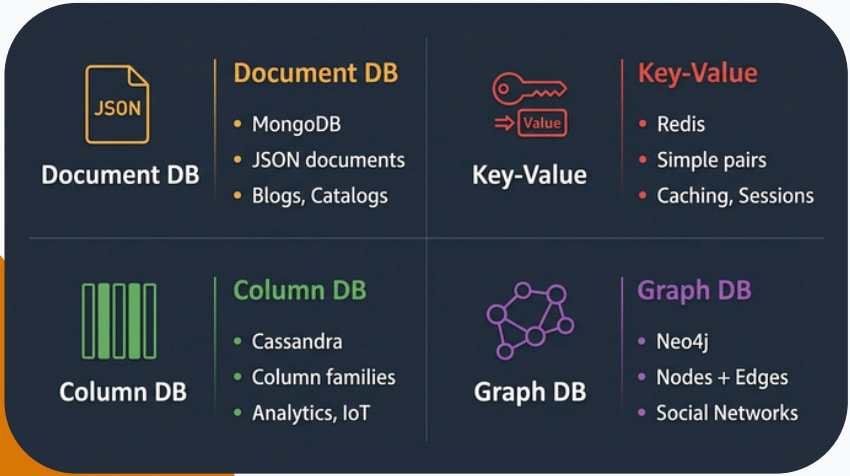
- Banking / financial apps
- E-commerce (products, orders, payments)

When NOT to Use

- Datastructurechangeseveryweek
- Needtodistributeacross100+serversglobally
- Storingunstructuredsensor/IoTdatastreams

NoSQL Databases — The 4 Sub-Types

Type 2 “Not Only SQL” — a whole family of databases built for flexibility and scale.



NoSQL — Strengths & Weaknesses

Strengths

- **Flexibleschema**—Noneed to define structure up front. Add fields anytime.
- **Horizontalscaling**—Addmoreservers(notbigger servers). Netflix handles 200M+ users this way.
- **Blazing fast** — Redis responds in **microseconds**.
- **Great for unstructured data**—Socialposts,sensor data, game states.
- **Developer friendly**—DatalookslikeJSON(you already know it!).

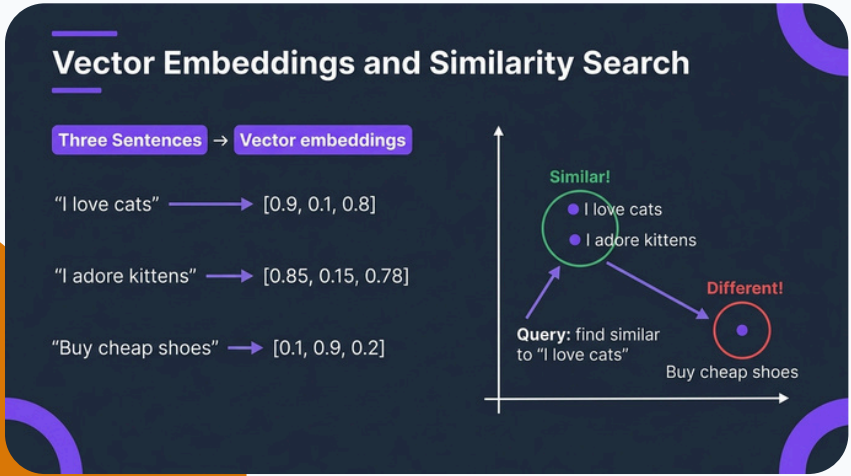
Weaknesses

- **NoJOINs**(usually)—Mustdenormalize(duplicate data).
- **No ACID** (usually) — Eventual consistency: data may be *slightly* out of date.
- **Nostandardlanguage**—EachDBhasitsownquery syntax.
- **Dataduplication**—Samedatastoredinmultiple places can get out of sync.
- **Hardertoenforcerules**—Nobuilt-inconstraintslike SQL.

When to choose NoSQL: Your data structure changes frequently, you need massive read/write throughput, or you're building a real-time system (chat, gaming, IoT).

Vector Databases — Built for AI

Type 3 The newest type. Stores meaning, not just text. Powers ChatGPT, Copilot, and RAG.

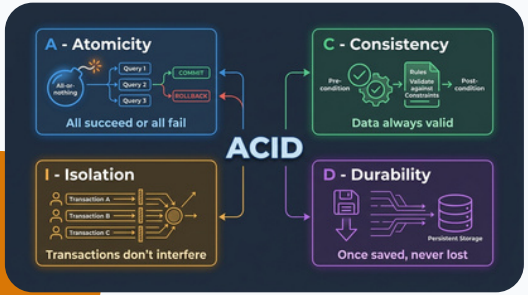


SQL vs NoSQL vs Vector — Full Comparison

Feature	SQL	NoSQL	Vector DB
Data model	Tables (rows & cols)	Documents / KV / Graph	Vectors (numbers)
Schema	Fixed (strict)	Flexible (schemaless)	Embedding dimensions
Query style	SQL language	DB-specific API	Similarity search (KNN)
Relationships	JOINs (powerful)	Embedded / manual	Not applicable
Scaling	Vertical (bigger server)	Horizontal servers)	Horizontal
ACID	✓ Full	Partial / None	No
Speed	Fast (indexed queries)	Very fast (reads/writes)	Fast (ANN search)
Best for	Structured + Relations	Flexible + Speed +Scale	AI + Semantic Search

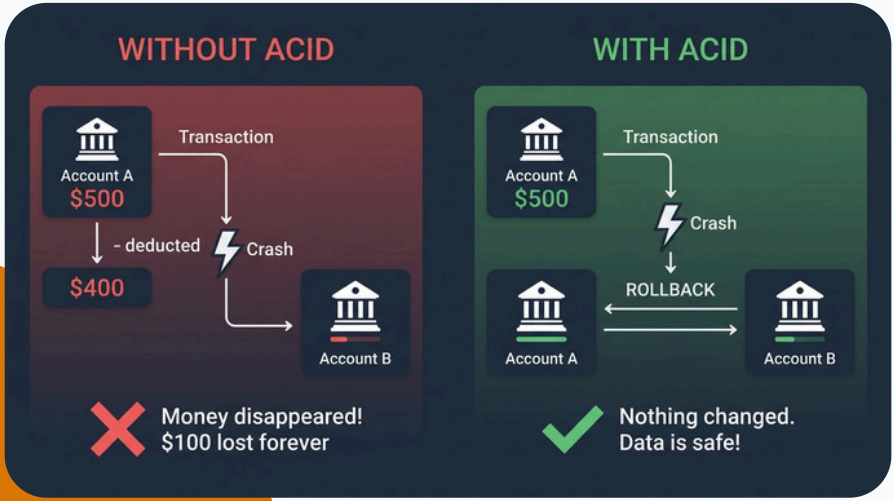
ACID Properties — Why Banks Trust SQL

Core Concept 4 guarantees that make your data bulletproof. Every SQL database supports these.



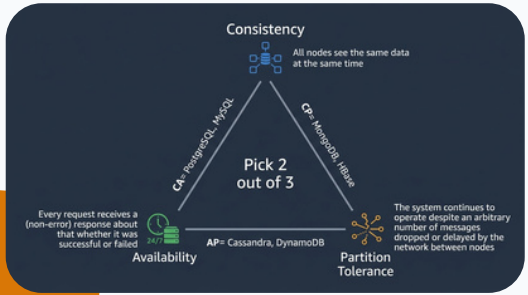
Letter	Meaning + Example
Atomicity	*All or nothing.* Transfer $100: both debit and credit happen, or neither does.
Consistency	*Rules always hold.* Balance can never go below zero.
Isolation	*No interference.* Two transfers at the same time don't corrupt each other.
Durability	*Once committed, permanent.* Even if the server explodes 1ms later.

ACID in Action — The Bank Transfer



CAP Theorem — The Impossible Triangle

Core Concept In a distributed system, you can only guarantee 2 out of 3. You must sacrifice one.



Property	Meaning
Consistency	Every read returns the **latest** write. All servers see the same data at the same time.
A vailability	Every request gets a response, even if some servers are down.
Partition Tolerance	System continues working even if the network between servers fails.

CA: PostgreSQL, MySQL *(single server — no partition)*
CP: MongoDB, HBase *(may reject writes to stay consistent)*
AP: Cassandra, DynamoDB *(always responds, but may show stale data)*

CAP in Real Life — Instagram vs Your Bank

Your Bank (Chooses Consistency)

Scenario: You transfer $500 to your friend.

Behavior: Both of you see the **exact same balance** at the same time. No stale data ever.

Trade-off: If the servers can't communicate (partition), the bank **rejects your transfer** rather than risk showing wrong data.

You see: "Service temporarily unavailable. Try again later."

This is CP — Consistency + Partition Tolerance.

Instagram (Chooses Availability)

Scenario: You post a photo. Your friend checks your profile.

Behavior: Your friend might **not see the photo for a few seconds**. Different servers sync at slightly different speeds.

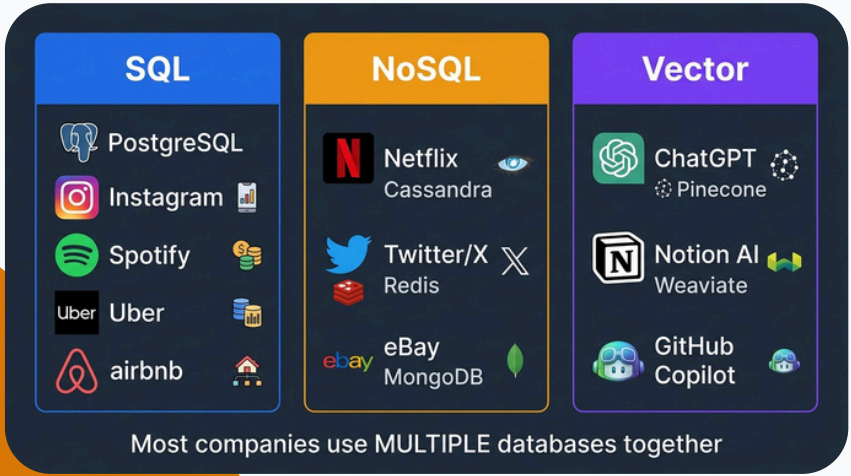
Trade-off: Instagram **always loads** (never shows "service unavailable"), even if data is 2 seconds behind.

You see: The post appears for everyone within 1–5 seconds.

This is AP — Availability + Partition Tolerance.

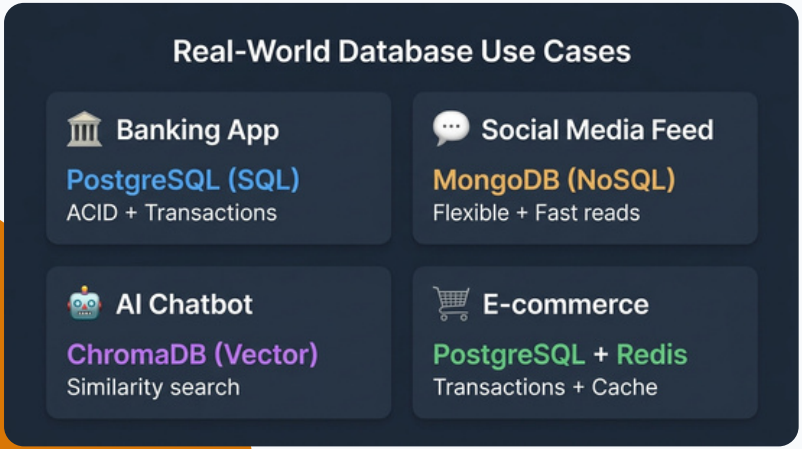
Key Insight: For a bank, showing wrong data = lawsuits. For Instagram, a 2-second delay = nobody cares. **Choose based on what your app can tolerate.**

Who Uses What? — Real Companies

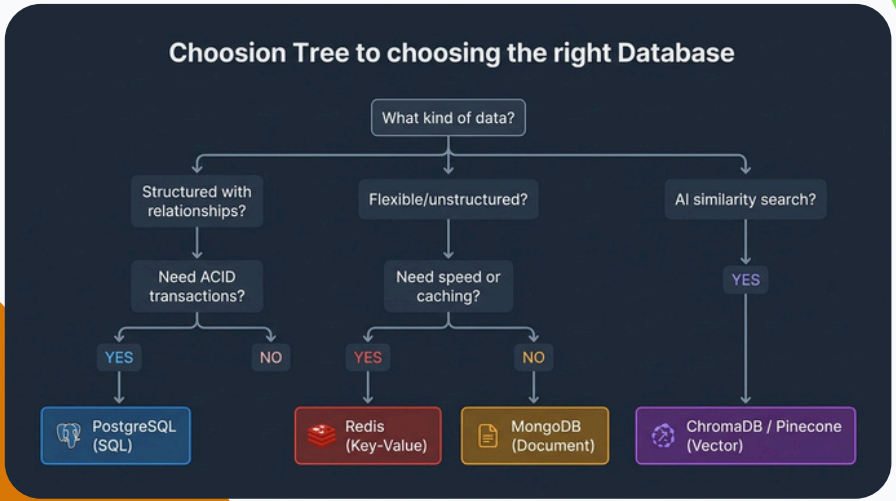


Scenario Challenge — Which Database Would You Pick?

Think! For each scenario, decide: SQL, NoSQL, or Vector? Then check the answer.



The Decision Flowchart — How to Choose



Why PostgreSQL First?

PostgreSQL is the Swiss Army Knife:

- **100% free & open source** — no license fees, ever
- **Full ACID** — trusted by banks and hospitals
- **SQLite universal** — works with MySQL, SQLite, etc.
- **JSON support** — can act like a document DB!
- **pgvector** — can do vector search too!
- **Used by:** Apple, Instagram, Spotify, Uber, Airbnb

Our Learning Path:

1. **DB-01:** Concepts + Types + ACID + CAP
2. **DB-02:** SQL — CREATE, INSERT, SELECT, WHERE
3. **DB-03:** Relationships — Keys, JOINs
4. **DB-04:** Advanced — Aggregations, Design
5. **FastAPI:** SQLAlchemy + PostgreSQL
6. **AI:** Vector DB (ChromaDB / pgvector)

Cheat Sheet — Everything on One Slide

3 Types

- **SQL**—Tables,JOINs,ACID
- **NoSQL**—Documents,speed, scale
- **Vector**—AI,meaning, similarity

When in doubt:
Start with **PostgreSQL**!

ACID

- **A**tomicity—Allornothing
- **C**onsistency—Ruleshold
- **I**solation—Nointerference
- **D**urability—Oncesaved,safe

= Safe data.
Banks, hospitals, e-commerce.

CAP

- **C**onsistency—Samedata everywhere
- **A**vailability—Always responds
- **P**artition—Survivessplits

= Pick 2 of 3.
Bank=CP,Instagram=AP.

Thank You